

⚠ Please do not reach out to reviewers directly; messaging them will not speed up your review.

Writing Oracle Solution

The oracle solution (`solution/solve.sh`) is an expert-authored script that reliably completes the task. It's used to verify that the task is solvable. For milestone tasks, the oracle solution lives per-milestone under `steps/milestone_N/solution/`: each milestone has its own `solveN.sh` (independently scoped to that milestone) plus a `solve.sh` wrapper that runs it. See the [Milestones page](#) for details.

Getting Started

Video Tutorial

Creating a solution/solve.sh



1.2×

9-min ⚡ 7 min 47 sec

What You'll Learn

- Structure of a good solution/solve.sh file
- How to transfer commands from interactive testing
- Best practices for deterministic solutions
- Common pitfalls to avoid



BASH

Copy

```
#!/bin/bash
set -e # Exit on error

# Step 1: Navigate to working directory
cd /app

# Step 2: Perform the task
sed -i 's/bug/fix/' main.py

# Step 3: Verify the fix
python -c "from main import process; assert process('test') == expected"

# Step 4: Any cleanup or final steps
echo "Task completed successfully"
```

Key Principles

1. Demonstrate Command Sequence

The solution should show the **steps to derive the answer**, not just the answer itself.

Good:

BASH

Copy

```
#!/bin/bash
# Find and fix the bug
grep -r "TypeError" /app/logs/ | head -1
# Found: main.py:42: TypeError: 'NoneType' object

# Fix the bug
sed -i '42s/data.process()/data.process() if data else None/' /app/

# Verify
python -m pytest /app/tests/ -v
```

Bad:**BASH** Copy

```
#!/bin/bash
# Don't just echo the answer!
echo "42" > /output/answer.txt
```

2. Must Be Deterministic

Running the solution multiple times should produce the same result.

Avoid:

- Random values without seeds
- Time-dependent operations
- Network calls to external services

3. Written By Human

The solution should be written by you, not generated by an LLM. Minimal LLM assistance for syntax is acceptable.

Advanced Patterns

Multi-Step Solutions

BASH

 Copy

```
#!/bin/bash
set -e

# Step 1: Set up environment
cd /app
source venv/bin/activate

# Step 2: Fix first issue
python -c "
import json
config = json.load(open('config.json'))
config['debug'] = False
json.dump(config, open('config.json', 'w'))
"

# Step 3: Fix second issue
sed -i 's/localhost/0.0.0.0/' server.py

# Step 4: Restart service
kill -f server.py || true
python server.py &
sleep 2

# Step 5: Verify
curl -s http://localhost:8080/health | grep -q "ok"
```

Using Python in Solution

BASH

 Copy

```
#!/bin/bash
cd /app

python << 'EOF'
import pandas as pd

df = pd.read_csv('/data/input.csv')
df['total'] = df['price'] * df['quantity']
```

```
df.to_csv('/data/output.csv', index=False)
EOF
```

Interactive Commands (solution.yaml)

For tasks requiring interactive tools like vim:

YAML Copy

```
# solution/solution.yaml
commands:
  - type: interactive
    command: vim /app/file.txt
    inputs:
      - "dd"      # Delete line
      - "i"       # Insert mode
      - "hello"   # Type text
      - "<Esc>"    # Exit insert
      - ":wq"     # Save and quit
```

Common Mistakes

Hardcoding Answers

BASH Copy

```
# WRONG: This just outputs the answer
echo "The answer is 42" > /output/result.txt

# RIGHT: This derives the answer
cd /app
python calculate.py > /output/result.txt
```

Non-Deterministic Solutions

BASH Copy

```
# WRONG: Random without seed
python -c "import random; print(random.choice([1,2,3]))"

# RIGHT: Deterministic
python -c "import random; random.seed(42); print(random.choice([1,2
```

Missing Error Handling

BASH Copy

```
# WRONG: Silent failures
cd /nonexistent || true
do_something

# RIGHT: Fail fast
set -e
cd /app
do_something
```

Testing Your Solution

Run Locally

BASH Copy

```
# Enter the container
harbor tasks start-env -p <task-folder> -i

# Inside container, run your solution steps manually
# Verify each step works as expected
```

Run Oracle Agent

BASH Copy

```
harbor run -a oracle -p <task-folder>
```

The oracle agent should PASS. If it fails, either:

- Your solution has bugs
- Your tests are too strict
- The task has issues

Next Steps

- [Write tests](#)
- [Run oracle agent](#)

PREVIOUS

[Dockerfile Requirements](#)

NEXT

[Writing Tests](#)